

Alternativas a Supabase: MySQL/MariaDB propio y almacenamiento de fotos vía Nextcloud

Documento de referencia por si en algún momento hay que prescindir de Supabase.

Responde a dos preguntas: sustituir Supabase por MySQL/MariaDB autoalojado, y usar una carpeta compartida con Nextcloud como almacén de las fotos.

Inventario real de uso de Supabase en el proyecto (verificado por código)

- **Auth (GoTrue):** `src/lib/supabase/{client,server,admin}.ts` , `src/proxy.ts` , `src/features/auth/services/auth.ts` , y 8 servicios admin más usan `signInWithPassword` , `signUp` , `signOut` , `resetPasswordForEmail` , `updateUser` , `getUser` , `exchangeCodeForSession` , y la Admin API (`auth.admin.createUser` , `deleteUser` , `listUsers`) — esta última es imprescindible para "Reiniciar sistema", borrar participantes y el alta manual por email.
- **Postgres + RLS:** 3 tablas (`profiles` , `submissions` , `contest_config`) con 13 políticas RLS que son **la barrera de seguridad real** (documentado explícitamente en el propio código: las Server Actions confían en que RLS bloquee lo que no debería pasar). Además 5 funciones/triggers en PL/pgSQL: `is_admin()` , `prevent_role_self_escalation()` , `enforce_profile_rules()` , `enforce_submission_rules()` (ventana de fechas, límite de fotos, bypass de modo_pruebas / `service_role`), `handle_new_user()` .
- **Storage:** bucket privado `contest-photos` con sus propias políticas RLS sobre `storage.objects` . 10 puntos de uso en 8 ficheros (`upload` , `remove` , `download` , `createSignedUrl`) — todos pasan por ese único nombre de bucket.
- **No se usa:** Realtime, Edge Functions, RPC (`supabase.rpc`) — confirmado por búsqueda en todo `src/` . Esto simplifica el análisis: no hay que preocuparse por esas piezas.

Pregunta 1 — ¿Sustituir Supabase por MySQL/MariaDB en un contenedor propio?

Respuesta corta: es posible, pero no es "cambiar de base de datos" — es reescribir casi todo el backend. No se recomienda salvo que el objetivo sea específicamente dejar de usar Postgres (no solo dejar de depender de la nube de Supabase).

Motivo: Supabase no es solo Postgres con otro nombre. Aporta tres piezas que MySQL/MariaDB no tienen ni de lejos un equivalente directo:

1. **Sin Row Level Security.** MySQL y MariaDB no tienen RLS. Ahora mismo, quién puede leer/escribir cada fila lo decide la base de datos (13 políticas), no el código Next.js. Sin RLS, cada una de esas reglas hay que reescribirla a mano dentro de cada Server Action y cada ruta API — un cambio de arquitectura de seguridad completo, con riesgo real de dejar un agujero si se olvida un caso.
2. **Sin GoTrue (Auth).** Login, registro, reset de contraseña, sesiones por cookie (`@supabase/ssr`), y la Admin API para crear/borrar usuarios — todo eso hay que sustituirlo por otra solución (Auth.js/NextAuth, Lucia, o autenticación hecha a mano con bcrypt + JWT). Afecta a más de 15 ficheros.
3. **Sin PostgREST.** `supabase.from('tabla').select()` funciona porque Supabase genera una API REST automática respetando RLS. Con MySQL no existe eso: haría falta un ORM (Prisma/Drizzle) y escribir a mano cada consulta y cada comprobación de permisos que hoy hace RLS gratis.
4. **Triggers en otro dialecto.** Los 5 triggers/funciones PL/pgSQL habría que reescribirlos en el dialecto de MySQL (sintaxis de triggers distinta, sin `SECURITY DEFINER` , manejo de errores diferente).

En la práctica esto es un backend nuevo desde cero, no una migración. Para una app de este tamaño (concurso municipal) es un esfuerzo desproporcionado si el motivo real es solo "no depender de un proveedor externo".

Alternativa recomendada: autoalojar Supabase, no sustituirlo

Supabase es software libre. Toda su pila (Postgres + GoTrue + PostgREST + Storage) se puede desplegar en contenedores Docker **dentro de la propia infraestructura municipal** (el mismo CPD donde ya corre esta app), sin depender de supabase.com. Con esto:

- **Cero cambios de código.** El proyecto seguiría usando `@supabase/supabase-js` y `@supabase/ssr` exactamente igual, solo cambia `NEXT_PUBLIC_SUPABASE_URL` para apuntar

al Supabase propio en vez de a supabase.com.

- Se conservan las 13 políticas RLS, los 5 triggers y toda la lógica de negocio ya probada, sin reescribir nada.
- Elimina la dependencia real que preocupa (un servicio externo fuera de su control), sin el coste de reescribir auth + autorización + API.

Procedimiento si deciden autoalojar Supabase:

1. En el servidor (o uno nuevo), clonar `supabase/docker` desde el repo oficial de Supabase (`docker-compose` con Postgres, GoTrue, PostgREST, Storage, Studio, Kong).
2. Exportar el esquema actual: `supabase db dump` (o `pg_dump`) contra el proyecto cloud actual — trae las 3 tablas, las 13 políticas RLS y los 5 triggers tal cual.
3. Importar ese dump en el Postgres autoalojado.
4. Migrar los usuarios de `auth.users` (GoTrue permite exportar/importar, o se puede usar la Admin API para recrearlos si son pocos — en este proyecto, probablemente pocos participantes en la fecha de migración).
5. Migrar los objetos del bucket `contest-photos` (copia simple de ficheros, Storage de Supabase autoalojado usa el mismo formato).
6. Cambiar `NEXT_PUBLIC_SUPABASE_URL` , `NEXT_PUBLIC_SUPABASE_ANON_KEY` y `SUPABASE_SERVICE_ROLE_KEY` en el `.env` de producción al nuevo Supabase propio, y `docker compose build + up` de la app (mismo proceso que cualquier despliegue ya documentado en `DESPLIEGUE_PRODUCCION.md`).
7. Verificar login, subida, moderación y galería contra el nuevo Supabase antes de apagar el proyecto cloud.

Si aun así se decide ir a MySQL/MariaDB completo (alto esfuerzo, alto riesgo, no recomendado salvo motivo de peso), el orden de trabajo sería:

1. Elegir y montar la solución de Auth (NextAuth/Auth.js es lo más estándar en Next.js).
2. Diseñar el esquema en MySQL/MariaDB con Prisma o Drizzle como ORM.
3. Reescribir cada comprobación de RLS como código explícito en cada Server Action y ruta API (auditoría fila por fila de las 13 políticas actuales para no perder ninguna regla).
4. Reescribir los 5 triggers como lógica de aplicación (ej. `enforce_submission_rules` pasa a ser una función TypeScript llamada antes de cada `insert`).
5. Sustituir Supabase Storage (ver Pregunta 2 para el almacenamiento en sí).
6. Migrar los datos existentes (dump de Postgres → transformar → importar en MySQL, contraseñas de usuarios no son migrables sin más porque GoTrue las hashea con su propio esquema — probablemente forzar reset de contraseña a todos los usuarios).

7. Reescribir toda la capa de autenticación en `src/lib/supabase/*` , `src/proxy.ts` , `src/features/auth/*` y los ~15 ficheros de `src/features/admin/services/*` que usan la Admin API de Supabase Auth.

Pregunta 2 — ¿Carpeta compartida con Nextcloud para las fotos?

Respuesta corta: sí, es viable y de esfuerzo bajo/medio — y además resuelve un problema ya detectado (los backups de Supabase no incluyen Storage). A diferencia de la pregunta 1, esto es un cambio aislado: se puede hacer sin tocar Auth ni la base de datos, tanto si siguen en Supabase cloud, autoalojado, o incluso si algún día migran a MySQL.

Uso real de Storage en el proyecto (todo pasa por el bucket `contest-photos`):

- Subida: `submissions-client.ts` (participante sube su foto)
- Borrado: `submissions-client.ts` , `reset-actions.ts` , `participant-actions.ts` , `manual-submission-actions.ts`
- Descarga/URLs firmadas: `get-gallery.ts` , `get-winners.ts` , `get-my-submissions.ts` , `get-all-submissions.ts` , `zip/route.ts` (todas usan `createSignedUrl` porque el bucket es privado — sin eso, cualquiera con la URL vería fotos no aprobadas)

Hay dos formas de hacerlo, de menor a mayor esfuerzo (Opción A y B abajo). Antes de elegir, hay una decisión previa que condiciona ambas: **cómo llegan los ficheros hasta Nextcloud** — por API (WebDAV) o por carpeta compartida en disco. Esto depende de dónde esté Nextcloud respecto al servidor de la app:

Cómo copiar a Nextcloud: API WebDAV vs. carpeta compartida

Carpeta compartida (recomendado si Nextcloud vive en el mismo servidor/CPD, 10.158.0.3 u otra máquina con almacenamiento de red accesible desde ahí). Nextcloud incluye de serie la app "External Storage" (Ajustes → Administración → Almacenamiento externo), que permite dar de alta una carpeta local del propio servidor como si fuera una carpeta más dentro de Nextcloud — Nextcloud simplemente lee esos ficheros del disco, no hay sincronización ni copia interna de por medio. Desde la app, esto significa:

- La app escribe con `fs.writeFile` normal y corriente en una carpeta montada como volumen Docker (ej. `-v /datos/concurso-fotos:/app/fotos-backup`).
- Nextcloud, apuntado a esa misma ruta del disco, la muestra tal cual, al instante.

- Sin llamadas HTTP, sin tokens, sin gestionar reintentos por caída de red, sin límites de tasa de la API. Es el método que la propia documentación de Nextcloud recomienda para "grandes cantidades de ficheros que ya existen en el servidor" — que es exactamente este caso.
- Único requisito: que el proceso de Nextcloud (contenedor o servicio) y el proceso de esta app puedan ver la misma ruta de disco o del mismo NFS/SMB — algo típico si ambos corren en el mismo CPD.

API WebDAV (solo si Nextcloud está en otra infraestructura sin filesystem compartido, p. ej. otro servidor, otra red, o un Nextcloud como servicio externo). Nextcloud expone cada carpeta de cada usuario por WebDAV en

`https://<nextcloud>/remote.php/dav/files/<usuario>/<ruta>` . Para subir un fichero basta un `PUT` HTTP con Basic Auth; para borrar, un `DELETE` . Puntos a tener en cuenta:

- **Nunca usar la contraseña real de la cuenta.** Nextcloud permite generar una "contraseña de aplicación" en Ajustes → Seguridad, específica para esta integración, revocable sin afectar al login normal del usuario — es el método que Nextcloud recomienda para integraciones de terceros, en vez de la contraseña principal.
- En Node no hace falta librería especial: `fetch` con método `PUT` y las cabeceras de auth es suficiente (o el paquete `webdav` de npm si se quiere una API más cómoda con `createDirectory` , `putFileContents` , etc.).
- Al ser una llamada de red que puede fallar o tardar, **debe hacerse en segundo plano, nunca bloqueando la respuesta al participante que está subiendo su foto** — por ejemplo disparando la copia sin esperarla (`void copiarANextcloud(...)`) o con una cola simple, y registrando errores para reintentarlos, en vez de que un fallo de Nextcloud impida subir la foto a la web.

Recomendación: salvo que se confirme que Nextcloud corre en infraestructura separada, usar la carpeta compartida — es más simple, más fiable (sin dependencia de red) y es el propio método que Nextcloud aconseja para este escenario. Conviene confirmar primero dónde está desplegado el Nextcloud municipal antes de decidir.

Opción A (recomendada ya mismo): copia de seguridad en paralelo, sin tocar nada

No sustituye Supabase Storage — lo complementa. Cada vez que se sube o aprueba una foto, una tarea la copia también a una carpeta que Nextcloud tiene configurada como "almacenamiento externo" (carpeta local). Resuelve el hueco de backup ya anotado en la memoria del proyecto sin ningún riesgo para el flujo actual.

Procedimiento:

1. Montar un volumen Docker adicional en el contenedor de la app, apuntando a una carpeta del servidor que Nextcloud también vea (bind mount o carpeta de red).
2. En Nextcloud, dar de alta esa misma carpeta como "External Storage → Local" — así aparece en la interfaz de Nextcloud sin que la app tenga que hablar con la API de Nextcloud en ningún momento.
3. Añadir una copia a disco (`fs.writeFile`) justo después de cada subida exitosa en `submissions-client.ts` , y al aprobar/marcar ganadora en `moderation-actions.ts` / `premios-actions.ts` (opcional: solo copiar las aprobadas, para no acumular fotos rechazadas).
4. Supabase Storage sigue siendo la fuente de verdad para la app; la carpeta es solo copia de seguridad consultable/descargable desde Nextcloud.

Opción B: sustituir Supabase Storage por la carpeta compartida

Elimina la dependencia de Storage por completo, manteniendo Auth y base de datos en Supabase (o donde estén).

Procedimiento:

1. Montar el volumen Docker con la carpeta (igual que en la opción A), dada de alta en Nextcloud como almacenamiento externo local.
2. Sustituir los 10 puntos de uso de `supabase.storage.*` por sus equivalentes de `filesystem`: `upload` → `fs.writeFile` , `remove` → `fs.unlink` , `download` → `fs.readFile` .
3. **El punto delicado es** `createSignedUrl` (bucket privado + URL temporal autenticada) — el `filesystem` no tiene equivalente nativo. Hay que crear una ruta API propia (ej. `src/app/api/photos/[id]/route.ts`) que compruebe permisos (RLS ya no aplica a ficheros en disco, así que esta ruta debe repetir en código la misma comprobación que antes hacía la política de Storage: dueño de la foto, o aprobada/ganadora para el público, o admin) y sirva el fichero (`fs.createReadStream` • `NextResponse`). Sustituye a `createSignedUrl` en los 5 ficheros que lo usan.
4. Actualizar `GalleryGrid` , `WinnersGrid` , `ModerationTable` , `MySubmissions` para usar esa nueva URL (`/api/photos/[id]`) en vez del `signedUrl` que devuelve Supabase.
5. Migrar las fotos ya subidas: descargar todo el bucket `contest-photos` y copiarlo a la carpeta con la misma estructura de rutas (`{participant_id}/{uuid}.{ext}`), para no tener que tocar la columna `storage_path` de `submissions` .

Recomendación

- Si la preocupación es depender de la nube de Supabase (coste, gobernanza, que la web municipal no dependa de un tercero externo): **autoalojar Supabase** resuelve eso sin reescribir nada, es la opción de menor riesgo y menor esfuerzo con diferencia.
- Sustituir por MySQL/MariaDB solo tiene sentido si el objetivo es específicamente dejar de usar Postgres/RLS como tecnología — es un proyecto de reescritura de semanas, no un cambio de configuración.
- La carpeta compartida con Nextcloud (Opción A) es una mejora independiente y de bajo riesgo que se recomienda hacer en algún momento de todos modos, resuelva o no el resto — cierra el hueco de backup de fotos ya detectado, sin esperar a decidir nada sobre Supabase.